In a typical extension with a background page, the UI — for example, the browser action or page action and any options page — is implemented by dumb views. When the view needs some state, it requests the state from the background page. When the background page notices a state change, the background page tells the views to update.

The background page looks something like this in your manifest:

```
{
  "name": "extension name",
  ...
  "background _ page": "background _ page.html",
  ...
}
```

To speed up the browser startup, consider specifying the background permission.

You can communicate between your various pages using direct script calls, similar to how frames can communicate. The chrome.extension.getViews() method returns a list of window objects for every active page belonging to your extension, and the chrome.extension.getBackgroundPage() method returns the background page.

The following code snippet demonstrates how the background page can interact with other pages in the extension. It also shows how you can use the background page to handle events such as user clicks.

The extension in this example has a background page and multiple pages created (with chrome.tabs.create()) from a file named image.html.

```
//In the background page:
<html>
  <script>
    //React when a browser action's icon is clicked.
    chrome.browserAction.onClicked.
addListener(function(tab) {
      var viewTabUrl = chrome.extension.getURL('image.
html');
      var imageUrl = /* an image's URL */;

      //Look through all the pages in this extension to
find one we can use.
      var views = chrome.extension.getViews();
      for (var i = 0; i < views.length; i++) {
        var view = views[i];
```